AIC

2026 年第八届

全球校园人工智能算法精英大赛

算法主题赛（AI+开源）

ReproLens 开源 AI/ML 项目复现就绪度审计技术报告

参赛形式：个人参赛

申报方向：AI 开发工具与开源协作

作品名称：ReproLens / 复现镜

2026 年 10 月 30 日 30 日

目 录

作品简介……2
一、作品概述……3
二、需求分析与总体方案……5
三、AI 技术与开源及第三方资源选择……7
四、设计与实现过程……8
五、成果展示与效果验证……10
六、创新性与开放价值……12
七、合规、安全与风险说明……13
八、总结与展望……14
九、附录……14

作品简介

开源论文项目普遍宣称可复现，但依赖、数据、模型、配置和评测证据分散在 README、脚本与链接中，维护者难以自证，评审者也缺少统一核查入口。ReproLens 面向公开 GitHub AI/ML 项目，优先支持 Python 为主的论文代码仓库；固定 commit 后静态扫描数据、模型、实验、文档与许可证据，生成六维评分、N/A 适用性判断、README 声明-证据矩阵和证据附录，并直接列出具体缺口。系统可在隔离副本中生成 AutoFix 提案并复测，只保留能提高评分的修改，同时提供网页、JSON、Webhook、GitHub Action 与飞书自动通知。验证覆盖多个 AI 领域，并以 90 项测试保障稳定。

作品定位：给定一个公开 GitHub 仓库，回答它能复现什么、证据在哪里、还缺什么，以及下一步应补什么。

一、作品概述

（一）背景与问题

开源软件、模型和数据已经成为人工智能研发的重要基础设施，但“代码公开”并不等于“他人能够复现”。一个可复现的 AI 项目通常需要同时说明环境、数据、模型、实验配置、评测方式和许可边界。当前这些信息分散在 README、配置文件、脚本、论文链接和外部托管平台中，维护者和评审者往往需要逐项人工查找。

我们做 ReproLens 的初衷，是把“别人能否复现这个开源 AI 项目”从依赖个人经验的判断，变成一条可复核、可整改、可自动复测的证据链。维护者在发布前需要知道仓库还缺什么，研究者和评审者需要在固定版本上核对证据，贡献者需要知道最小修改从哪里开始；仅给出一个总分，无法支撑这些行动。

我们在调研和真实仓库审计中看到：依赖扫描器主要关注漏洞与版本风险，容器工具主要解决运行环境，仓库健康评分又很少解释“某个分数为什么低”。这些工具分别覆盖了复现链条的一段，却尚未把“项目声明—仓库证据—缺口—整改—复测”连成可追溯流程。

（二）现有工具已经做了什么，仍然缺什么

为了避免做成“又一个通用代码扫描器”，我们先按复现链路梳理了四类直接工作与两类相邻工具。表 1 对比它们的成熟能力和仍未被覆盖的关键问题，并据此确定 ReproLens 的功能边界。

工具方向	已经具备的能力	仍未解决的关键问题
论文复现评测 CORE-Bench、PaperBench、REPRO-Bench、ReproRepo	评估智能体或研究者复现论文、仓库的能力，已形成 benchmark、标注和评价指标。	主要回答“能否发现问题、复现结果如何”，不面向维护者给出最小整改，也不提供修复后复测与 CI 闭环。
论文 artifact 评审 ArtifactCopilot、ArtifactGuide、LLM artifact evaluation	依据 ACM badge 或会议规范检查 artifact，辅助评审员形成结论。	场景偏论文附件和安全论文，规则与 badge 绑定，难以覆盖 LLM/RAG/Agent 的版本、索引、轨迹等风险。
环境打包与工作流 ReproZip、Popper、MLCube、Whole Tale	封装依赖、容器和实验流程，提高作者侧环境的可移植性。	要求作者主动采用；能固定运行环境，但不能审计任意第三方仓库是否具备完整复现证据。

工具方向	已经具备的能力	仍未解决的关键问题
实验管理与版本控制 DVC、MLflow、Sacred、Kedro	支持数据、模型、参数、指标和流水线记录，适合项目内部研发管理。	记录的是作者主动登记的内容；不解决第三方仓库的证据发现、README 声明核验和整改优先级。
代码质量、安全与依赖 CodeQL、SonarQube、Dependabot、OpenSSF Scorecard	发现代码缺陷、安全漏洞、依赖更新和仓库健康问题。	检查正确性与供应链风险，不回答模型版本、prompt、采样参数、RAG 索引和 Agent 轨迹是否可复现。
人工 checklist NeurIPS/MICCAI checklist、metacheck	以人工方式统一评审口径，适合论文投稿和评审流程。	依赖专家经验、耗时且结果难复测；证据分散，无法直接形成代码仓库的整改任务。

表 1 代表性工具能力与未覆盖缺口

综上，现有工具并不是“没有能力”，而是能力停留在复现链的不同片段：有人帮助运行，有人帮助记录，有人帮助评审，也有人帮助找代码问题，但很少有人面向仓库维护者同时回答“声称了什么、证据在哪、先改什么、改完是否真的更好”。这正是 ReproLens 的切入点。

因此，我们没有重复实现依赖扫描、漏洞发现或环境打包，而是集中完成五项组合创新：

- 仓库优先：直接审计公开 GitHub 或本地 AI/ML 项目，重点覆盖 LLM/RAG/Agent 场景，固定 commit，不要求作者迁移到新平台。
- 风险补全：在环境、数据、模型、实验、文档五维之外，增加模型/API、prompt、采样参数、tokenizer、RAG 索引、Agent 轨迹、污染和成本八项 LLM 专项检查。
- 证据闭环：把 README 声明映射到检查项，输出“已验证、部分验证、未验证、不适用”和可回查的证据编号。
- 整改复测：在隔离副本中生成最小修复候选，修复后重新审计，只保留真正提高评分的修改。
- 开放接入：用可选 LLM 完成证据约束的语义映射，异常时回退确定性规则，并通过 CLI、网页、JSON/Webhook 和 GitHub Action 进入真实协作流程。

（三）目标、赛题方向与作品形态

本作品申报“AI+开源”主题赛方向二“AI 开发工具与开源协作”，并融合方向三“开源项目改进与生态贡献”。申报主线是开发工具，因为 ReproLens 面向代码、文档、测试、数据与项目协作环节，为开源项目提供自动审查、证据追踪和整改建议；其评测与生态贡献能力作为辅助价值呈现。

目标用户包括开源 AI 项目维护者、科研论文作者、比赛评审与课程教师、开源贡献者以及需要在 CI 中设置质量门禁的团队。作品形态为浏览器应用、命令行工具、GitHub Action 和标准化 JSON/Webhook 接口的组合，而不是只提供一次性演示页面。

（四）核心亮点

- **证据优先：每个结论都关联证据编号、文件位置和原文，避免只给出不可解释的分数。**
- **适用性判断：先识别项目类型，再判断检查项是否适用；没有适用检查项的维度显示 N/A，不按 100% 或 0% 误导读者。**
- **声明-证据矩阵：把 README 中的能力声明映射到检查项，区分已验证、部分验证、未验证和不适用。**
- **闭环整改：AutoFix 只在隔离副本中生成候选修改，复测后保留真正提高评分的文件，原仓库保持不变。**
- **可泛化与可集成：规则不使用固定仓库名，支持公开 GitHub 仓库；优先覆盖 Python 为主的 AI/ML 项目，并输出 score.json、Webhook、GitHub Action 与飞书通知结果。**

二、需求分析与总体方案

（一）需求分析

通过对公开 AI 项目仓库的实际扫描，可以发现复现问题并不只是“有没有 requirements.txt”。论文项目可能提供数据集链接却缺少数据许可，Agent 框架可能不拥有自己的数据集，视频项目则可能依赖外部模型权重。工具必须同时理解项目类型、证据类型和适用边界。

目标对象	现有做法	主要痛点	ReproLens 的作用
项目维护者	人工检查 README 和目录	不知道评分低的具体原因	生成缺口清单和最小整改建议

目标对象	现有做法	主要痛点	ReproLens 的作用
论文作者与评审	阅读论文、README 和脚本	声明与仓库证据脱节	形成声明-证据矩阵和证据附录
开发团队	运行测试、人工汇总结果	结果难比较、难接入 CI	输出固定格式 JSON、Webhook 与 Action
开源贡献者	逐仓库熟悉结构	上手成本高、证据定位慢	用证据编号定位文件和原文

表 2 主要用户需求与对应能力

（二）总体方案

系统采用“输入层—来源固定—证据抽取—证据约束分析—分层输出”的结构。网页端接收公开仓库地址后，解析 GitHub 或 GitHub Pages 来源，固定 commit SHA，再进行浅克隆或归档回退。扫描器读取 README、依赖文件、配置、脚本、许可证和外部数据/模型链接，规则引擎按项目类型判断检查项是否适用。

图 1 ReproLens 总体系统架构

分析结果分为四类输出：可视化 HTML 报告、可被外部程序读取的 score.json、隔离副本中的 AutoFix 提案，以及用于 CI 门禁或外部系统的 Webhook。报告首页以总分环形图、六维雷达图、声明状态环图和证据覆盖率呈现整体结论，后续表格保留证据编号、文件和原文，形成“先看结论、再查证据”的阅读路径。

（三）可行性与成本

项目使用 Python 3.11 及以上版本，核心运行不依赖大型模型或 GPU，常规模块以标准库为主。公开仓库分析可在普通笔记本或一台低成本云服务器上完成；只有启用可选的 LLM 语义映射时才需要外部模型 API。当前公网实例已经部署在单台云服务器上，普通访问不需要用户安装环境。

成本控制策略包括：默认不执行仓库代码；对大型仓库使用浅克隆和归档回退；将分析拆分为可复用的扫描、评分和报告模块；AutoFix 只在临时副本中运行。当前版本的主要限制是私有仓库无法访问、超大仓库可能超时，以及静态证据无法证明完整运行必然成功，这些边界已在报告和文档中明确说明。

三、AI 技术与开源及第三方资源选择

（一）AI 技术作用

ReproLens 的基础评分采用确定性规则，保证同样输入在无外部模型时仍能重复生成结果。系统默认使用仓库自带的本地开源语义模型 ReproLens-SmallSemantic-v1，把 README 自然语言声明映射到固定检查项；外部 LLM 是可选后端，默认关闭。模型只负责语义候选，最终结果必须通过检查项目录、证据编号和结构校验；本地模型不可用或校验失败时自动回退到规则模式。

这种设计把 AI 放在“理解文本和辅助归纳”的位置，而不是让模型直接决定分数。报告中的每个结论都回到证据位置和原文，便于人工复核。对于公开论文仓库，系统优先识别真实数据集链接、数据下载命令、数据许可和模型来源；arXiv、DOI 或论文徽章只作为论文来源，不会替代数据证据。新增证据相关性规则后，依赖清单、数据链接、项目 LICENSE、通用 faiss 依赖和 logger 中的 run_id 都不会被当成不相关条件的证明。

图 2 数据证据识别优先级与误报抑制机制

（二）开源及第三方资源选择

资源类别	使用方式	许可与限制	合规状态
Python 3.11+	运行核心程序	PSF License	保留许可声明
公开 GitHub 仓库	分析对象与证据来源	各仓库自身许可	不重新分发完整代码
GitHub Actions	自动运行与报告归档	GitHub 服务条款	按条款使用
可选 LLM API	可选语义映射，默认关闭	服务商条款与隐私政策	操作者自行配置并审核
ReproLens 自有代码	开放源代码与报告格式	MIT License	保留版权与许可声明

表 3 开源及第三方资源选择与合规状态

（三）自主开发与权利边界

项目自主完成的部分包括仓库来源解析、证据抽取、项目类型识别、六维评分、N/A 适用性判断、README 声明-证据矩阵、可视化报告、AutoFix 隔离复测、JSON/Webhook 接口和自动化测试。第三方仓库仅作为被分析对象，系统

不打包、不重新分发其完整代码；报告只保留完成证据核查所需的有限原文片段和文件位置。

使用生成式 AI 辅助编码、文案整理和测试设计时，所有输出均经过人工审查、实际运行和来源核查。项目自身的开放许可证为 MIT，第三方名称、论文、数据集和模型仍归原权利人所有。若部署环境启用 LLM API，数据处理边界由部署者根据服务条款和隐私要求确认。

（四）AI 与开源融合方式

ReproLens 的 AI 与开源融合不是简单调用模型接口，也不是对现有项目更换界面。开源仓库提供可核验的事实证据，规则引擎负责可重复的判断，LLM 只在证据范围内补充语义理解。最终产物既是可浏览的报告，也是可被其他开源项目复用的 JSON、Webhook 和 GitHub Action 工作流。

四、设计与实现过程

（一）核心功能与流程

模块	输入	处理	输出
来源解析	GitHub URL / Pages	解析仓库、固定 commit、浅克隆或归档回退	可追溯的项目快照
证据扫描	项目文件与 README	识别依赖、数据、模型、实验、文档、许可和声明	带编号的证据清单
评分与适用性	证据清单与项目类型	六维评分、LLM 专项检查、N/A 判断	分数、维度和状态
报告与矩阵	评分与证据	生成可视化报告、声明-证据矩阵、缺口清单	HTML / Markdown / JSON
AutoFix	原始仓库与评分结果	隔离副本生成候选修改，反复扫描复测	有效修改、丢弃记录和前后对比
集成接口	score.json	Webhook、CI 门禁、GitHub Action	外部系统可消费的结果

表 4 核心功能模块与输入输出

（二）关键实现

1. 固定版本与来源回退。网页分析先解析公开 URL，固定 commit SHA 后再扫描；当浅克隆失败时回退到 GitHub 归档，避免报告随着默认分支变化而失去可比性。

2. 通用证据抽取。扫描器根据文件类型、目录结构和文本模式提取证据。数据证据优先使用 Hugging Face、Kaggle、Google Drive 等真实数据集入口，其次识别 hf download、load_dataset 等可执行命令，再识别数据许可和文字说明；论文链接单独归档，不冒充数据证据。

3. 适用性优先于分数。系统先判断 SDK、Agent 框架、应用、数据集项目和模型无关项目等类型，再决定数据、模型和实验检查项是否适用。N/A 维度不计入总分，也不会显示成 100%。

4. 证据相关性评分。规则引擎按环境、数据、模型、实验、文档和 LLM 特有风险六类维度计算得分。每条结论只附支持当前检查项的证据：依赖清单不能代替版本锁定，数据链接不能代替下载或预处理脚本，项目 LICENSE 不能代替数据或模型许可。每个判定同时保存证据编号、文件路径、行号和摘要，便于评委或维护者逐项复核。

5. 声明-证据矩阵。从 README 抽取能力声明，将其映射到可验证检查项，输出已验证、部分验证、未验证和不适用四种状态，并把缺口直接列在声明旁边。

6. AutoFix 隔离复测。AutoFix 只在 staging 副本中生成候选文件，每次加入候选后重新扫描和评分；只有能提高评分的修改才保留，原始仓库通过指纹校验保持不变。生成模板中的 TODO 不会被当作已验证证据。

7. 可视化与外部接口。报告使用原生 SVG/CSS 绘制总分环、雷达图、声明环和覆盖率条，不依赖外部前端 CDN；同时提供 score.json、Webhook、GitHub Action 和服务器端飞书通知，适配本地使用、在线演示与团队协作。

（三）团队分工与过程记录

本项目为个人组队参赛。参赛者独立完成选题、需求分析、系统设计、代码实现、测试、部署和技术报告整理。开发过程中使用生成式 AI 辅助代码检查、测试补充、界面文案和文档结构整理，所有 AI 生成内容均经过人工审查、实际运行和来源核对，未直接采用未经核验的结论。

开发过程分为四个阶段：第一阶段完成仓库扫描、规则评分和基础报告；第二阶段加入证据编号、声明矩阵、N/A 适用性和可视化；第三阶段完成网页分析、AutoFix、JSON/Webhook 与 GitHub Action；第四阶段使用多类真实仓库做泛化测试，修复论文链接误识别、空维度 100% 和标题可读性等问题，并完成 90 项自动化测试。

五、成果展示与效果验证

（一）成果展示

作品提供公网网页入口、开源代码仓库和可直接运行的本地命令行。评委可以输入任意公开 GitHub 仓库地址，等待系统生成固定 commit 的审计报告。

开源代码：https://github.com/czkPK/reprolens

公网演示：http://182.92.232.168:8080/

建议现场演示流程：

- **步骤 1：打开公网首页，粘贴公开 GitHub 仓库地址并提交分析。**
- **步骤 2：等待任务完成后，查看总分、六维雷达图、声明状态和证据覆盖率。**
- **步骤 3：进入完整报告，打开检查项证据、声明矩阵和证据附录，核对文件、行号与原文。**
- **步骤 4：打开 AutoFix，比较修改前后评分，并确认原仓库指纹未变化。**
- **步骤 5：下载 score.json，或通过 Webhook 页面确认外部系统收到了结构化结果。**
- 步骤 6：如果当前仓库在通知白名单中，服务器会把 score.json 摘要转换成飞书消息并自动发送；Webhook 只保存在服务器环境变量中，网页前端不会暴露地址。

图 3 ReproLens 公网审计报告总览（真实仓库 smolagents）

（二）验证方法

验证分为四层：第一，90 项自动化测试覆盖扫描、规则、评分、证据相关性、声明矩阵、报告、Webhook、GitHub Pages 解析、N/A、规则回退和飞书通知适配；第二，使用多个不同领域的论文官方仓库完成端到端分析；第三，通过 HTTP 检查确认首页、报告、JSON 和 Webhook 可访问；第四，对典型错

误做回归测试，例如 arXiv 链接不能替代数据集链接、数据集链接不能替代数据脚本、不适用维度不能显示为 100%、没有适用检查项时总分不除以零。

可直接复核的公网接口包括：

- **健康检查：http://182.92.232.168:8080/api/health 返回服务状态。**
- **结构化结果：每份报告的 score.json 可被外部程序读取，并保留维度、检查和证据编号。**
- **Webhook 看板：http://182.92.232.168:8080/webhook-demo 展示外部系统实际收到的 JSON。**
- **AutoFix 记录：每次修改前后评分、保留文件和丢弃原因均写入 autofix.html 与 autofix.json。**

图 4 AutoFix 前后评分对比

（三）验证结果

领域	论文官方仓库	分数	关键验证结果
自然语言处理	google-research/bert	71.4	引用 BERT checkpoint，LLM 专项检查为 N/A
语音识别	openai/whisper	79.4	从模型源码识别权重 URL，LLM 专项检查为 N/A
医疗影像	Project-MONAI/MONAI	69.7	通用 AI 框架不再误判为 LLM 应用
多模态	openai/CLIP	81.1	图像检索不误判 RAG，模型说明与权重来源分开
具身智能与机器人	facebookresearch/habitat-lab	76.1	下载命令和数据许可正确，协议 PDF 不算模型权重
时间序列	zhouhaoyi/Informer2020	71.0	代码许可证不再冒充数据许可
计算机视觉	google-research/vision_transformer	83.2	正确识别 ViT 权重地址，LLM 专项检查为 N/A

表 5 七个跨领域论文仓库验证结果（结果为本项目实际运行记录）

跨领域测试发现并修复了四类通用误报：非 LLM 项目被误判为 LLM、图像检索被误判为 RAG、代码或模型许可证被误判为数据许可、博客和协议 PDF 被

误判为模型权重。修复后，七个仓库的 LLM 专项检查全部正确显示为 N/A，数据与模型证据只引用真实数据链接、许可和 checkpoint。

VRMDiff 回归案例进一步验证了论文链接处理：arXiv 徽章只作为论文来源，系统改为引用 Hugging Face 数据集链接、load_dataset 加载命令和 CC BY-NC 4.0 数据许可。上述结果说明同一套规则可以在不使用固定仓库名的前提下跨领域工作。

AutoFix 示例中，原始项目的静态评分由 49.1 提升至 68.4；真实 VRMDiff 运行中，评分由 41.7 提升至 60.7。该结果只表示隔离副本中的静态证据改善，不表示完整运行已经成功，报告仍会保留尚未满足的检查和人工复核要求。

六、创新性与开放价值

（一）创新性

- **从“打分”转为“证据链”：报告不仅给出总分，还展示每条证据的文件、行号、原文和缺口，方便复核。**
- **从“统一检查”转为“适用性判断”：先识别项目类型，再决定检查项是否适用，避免把模型无关框架误判为缺少模型卡。**
- **从“结果页”转为“闭环流程”：声明矩阵、缺口诊断、AutoFix 和复测形成连续闭环，而不是只输出一份静态清单。**
- **从“特定样例”转为“通用规则”：数据证据按真实链接、可执行命令、许可说明和文字线索分级，论文链接不再误当数据证据。**

与只检查依赖或漏洞的工具相比，ReproLens 关注数据、模型、实验和文档之间的证据关系；与通用 AI 对话工具相比，ReproLens 的结论受证据编号和规则约束，能够被外部系统复核。项目的创新点在于把复现判断变成一套可追溯、可自动复测的工程流程。

（二）开放成果与复用方式

项目代码以 MIT License 开放在 GitHub，报告格式、评分 JSON、Webhook、GitHub Action 和飞书通知适配均可被其他项目复用。使用者可以把评分 JSON 送入 CI 门禁，把 Webhook 接入外部看板，或直接在 Action 中为开源仓库生成可下载的审计报告。当前开放内容不包含第三方仓库的完整代码或受限数据，只保留完成核查所需的证据引用。

（三）维护计划

后续计划包括：扩充多语言 README 和不同领域论文仓库的测试集；增加 PDF 论文与论文项目之间的证据关联；完善模型卡、数据集卡和许可条款识别；提供更细粒度的 CI 规则配置；根据真实使用者反馈改进报告可读性和误报解释。现阶段优先保证规则透明、证据可追踪和结果可重复，再逐步扩展覆盖范围。

七、合规、安全与风险说明

（一）开源与第三方许可

ReproLens 自有代码采用 MIT License。被分析仓库、数据集、模型、论文和第三方服务仍适用各自许可或服务条款。系统不把“公开可访问”视为当然可再分发，不在报告中复制完整仓库，只摘录完成证据核查所需的有限内容并标注来源位置。部署者在启用外部 LLM API 前，应核对服务商的数据处理、隐私和内容安全条款。

（二）数据、隐私与 AI 输出安全

默认模式只处理公开 GitHub 仓库，不执行仓库代码，不收集选手或用户的私有数据。AI 语义映射是可选功能，输出必须通过证据编号和结构校验，无法验证的候选会被丢弃并回退到规则模式。模型可能产生误判，因此报告把 AI 输出定位为辅助解释，而不是不可质疑的最终裁决。

（三）风险与控制措施

风险	可能影响	控制措施
论文链接被误当数据证据	数据维度虚高	证据分级、arXiv/DOI 单独标记、回归测试
静态扫描误报或漏报	评分与事实不一致	证据编号、原文引用、人工复核和 N/A 边界
大仓库扫描超时	无法生成报告	浅克隆、归档回退、文件和时间限制
外部 LLM 输出不稳定	解释偏离证据	默认规则回退、输出结构校验、证据编号验证
第三方许可不一致	开放成果合规风险	仅引用必要证据、保留来源、部署者确认服务条款

表 6 主要风险与控制措施

（四）技术限制与适用边界

ReproLens 当前证明的是“静态复现就绪度”，不是“项目一定可以成功运行”。真实运行还受到网络、硬件、数据授权和外部模型服务影响。当前优先支持 Python 为主的 AI/ML 论文代码仓库；非 Python、模型无关 SDK、私有资源或超大仓库可能只覆盖通用检查，部分项目类型会显示 N/A。对于这些情况，报告应被视为初筛结果，需结合人工复核和受控执行。

八、总结与展望

ReproLens 将开源 AI 项目的复现问题拆成可检查的证据链，完成了从公开仓库分析、证据抽取、适用性判断、评分和声明矩阵，到可视化报告、AutoFix 复测、JSON/Webhook 和 GitHub Action 的完整闭环。系统不依赖固定仓库名，已经在自然语言、语音、医疗影像、多模态、具身智能、时间序列和计算机视觉七个领域完成真实运行验证。

当前版本的不足是静态分析不能替代真实执行，部分自然语言声明仍需人工判断。下一阶段将扩大跨领域测试、加强论文与仓库事实的关联、完善数据/模型许可识别，并在不牺牲可追溯性的前提下降低误报。项目将继续以 MIT License 开放核心代码和报告格式，方便其他团队复用和二次开发。

九、附录

（一）开源及第三方资源使用清单

序号	资源名称及类型	版本/来源	许可证/授权	使用方式	关键义务/限制	合规状态
1	Python 运行时	3.11+；python.org	PSF License	运行核心程序	保留许可声明	合规
2	公开 GitHub 仓库	分析时固定 commit	各仓库自身许可	只读取并生成证据引用	不重新分发完整代码	按仓库许可复核
3	GitHub Actions	GitHub 服务	服务条款	自动化与报告归档	遵守平台使用条款	合规
4	OpenAI 兼容 LLM API	可选；默认关闭	服务商条款	语义映射辅助	部署者确认隐私与内容安全	条件合规

序号	资源名称及类型	版本/来源	许可证/授权	使用方式	关键义务/限制	合规状态
5	ReproLens 自有代码	v0.1.0；GitHub	MIT License	开放源代码与报告格式	保留版权与许可文本	合规

表 7 开源及第三方资源使用清单

（二）链接与运行说明

代码仓库：https://github.com/czkPK/reprolens

公网演示：http://182.92.232.168:8080/

赛题页面：https://www.aicomp.cn/tracks/tracks-5/4924.html

本地运行：在仓库目录执行 `python -m repolens.web --open`，或直接双击仓库中的启动脚本；网页端粘贴公开 GitHub 仓库地址即可开始分析。

（三）测试记录

截至本报告生成时，仓库内共执行 90 项自动化测试，覆盖扫描、规则、评分、本地语义映射、证据相关性、声明矩阵、报告渲染、AutoFix、JSON、Webhook、GitHub Pages 解析、N/A 渲染、飞书通知适配、论文链接与真实数据集链接区分等场景。代表性真实仓库结果见表 5，公网报告与 JSON 均可通过作品首页访问。

（四）参考资料

1. 2026 AIC·AI+开源算法主题赛赛题及竞赛规则，https://www.aicomp.cn/tracks/tracks-5/4924.html
2. 跨领域论文与官方仓库清单，benchmark/paper_repos.json
3. 跨领域验证记录，docs/跨领域泛化验证-2026-10-28.md
4. ReproLens 开源代码仓库，https://github.com/czkPK/reprolens
5. ReproLens 公网演示，http://182.92.232.168:8080/
6. 2026 AIC AI+开源技术报告参考大纲，大赛组委会发布文件。
7. ReproRepo: A Benchmark for Repository-Level Reproduction Auditing, arXiv:2606.18237.
8. ArtifactCopilot / ArtifactGuide: LLM-assisted artifact evaluation, arXiv:2602.02235.
9. Agentic Replication Package Quality Evaluation, arXiv:2606.02006.
10. CORE-Bench、PaperBench、REPRO-Bench: 复现智能体与论文复现评测基准.
11. ReproZip、Popper、MLCube、Whole Tale、DVC、MLflow、OpenSSF Scorecard 官方文档与项目主页.

（五）独立复核与复现步骤

评委可按照以下顺序独立复核核心功能、实验数据和开放成果：

- 运行 python -m unittest discover -s tests -v，确认当前 90 项自动化测试通过。
- 运行 python -m repolens audit examples/demo-llm-app --out-dir reports/demo-llm-app --autofix --explain，检查 report.html、claims.md、score.json 与 autofix/autofix.json。
- 运行 python -m repolens.web --open，粘贴公开 GitHub 仓库地址，核对 commit 固定、证据定位与报告生成。
- 运行 python scripts/fetch_benchmark_repos.py --name mem0 --name haystack 后执行 python -m repolens.benchmark，复核开发集与 holdout 指标。
- 检查 LICENSE、CHANGELOG.md、CONTRIBUTING.md、SECURITY.md 与《开源及第三方资源使用清单》，核对开放边界和维护计划。

上述材料均可从开源仓库与公网演示入口访问；报告中的评分和结论均为固定 commit 下的静态复现就绪度结果，不替代真实训练或推理验收。